# Programming for Data Science

# Git and Github

Phạm Trọng Nghĩa
ptnghia@fit.hcmus.edu.vn

# Student download

- 11-DemoGit.html

# Content

- Introduction to Version Control
- Introduction to Git
- Git - Basic usage
- Github

# Introduction to Version Control

# What is Version Control

- The Problem We've All Faced

```
project_final.ipynb
project_final_v2.ipynb
project_FINAL_ACTUALLY.ipynb
project_final_for_real_this_time.ipynb
```

- Without version control:
  - ❌ Lost experiment results and hyperparameters
  - ❌ Can't reproduce past models
  - ❌ Collaboration nightmares
  - ❌ Fear of breaking working code
  - ❌ No idea what changed when

# What is Version Control

- **A Time Machine for Your Code**
- **Version Control Systems (VCS)** track changes to files over time
- **Key Benefits:**
  - 📜 **Complete history** of all changes
  - 👥 **Seamless collaboration** with team members
  - ⏮️ **Revert** to any previous state
  - 🔍 **Track** who changed what, when, and why
  - 🔀 **Experiment** safely without breaking production code
  - 📊 **Reproduce** any analysis or model

# What is Version Control

**Perfect for Data Science:**

- Track experiments and their results
- Manage different versions of models
- Collaborate on ML pipelines
- Document your analytical decisions

# Version Control Systems Evolution

- **1. Manual Copying**
  - Many people's version-control method of choice is to copy files into another directory

Example:

*analysis_v1/, analysis_v2/, analysis_final/*

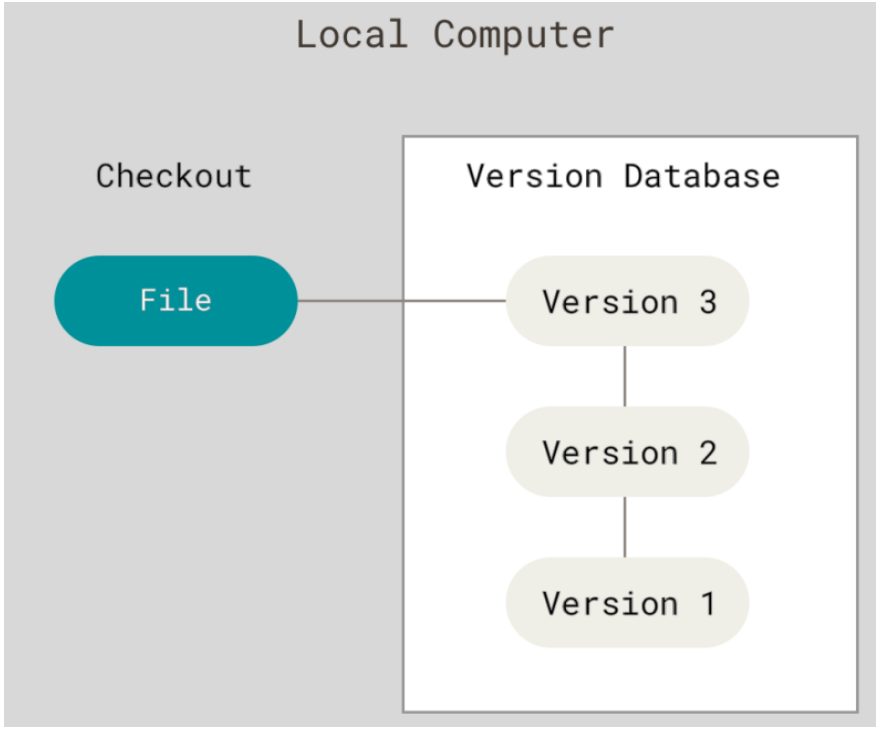
  - Error-prone, takes disk space
  - No meaningful change tracking

# Version Control Systems Evolution

- **2. Local VCS** (RCS)
  - ✓ Basic change tracking on your computer
  - ❌ No collaboration support

# Version Control Systems Evolution

- Ex: RCS
- Keeping **patch sets** (the differences between files) in a special format on disk
- Can then re-create what any file looked like at any point in time by adding up all the patches

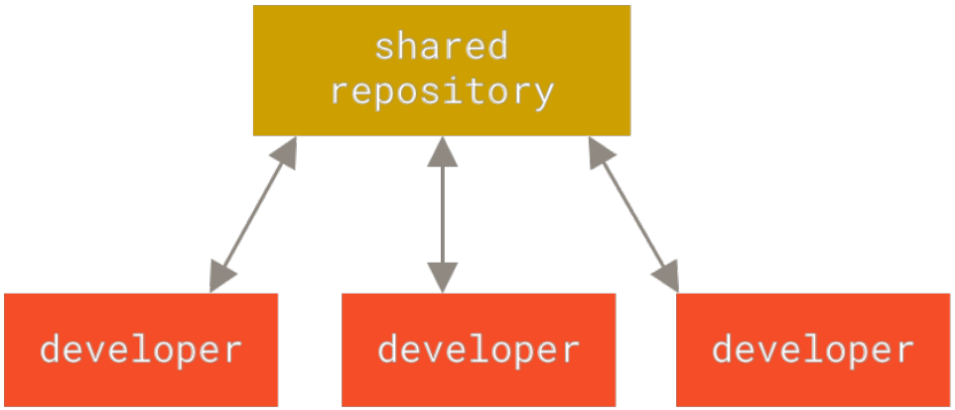


# Version Control Systems Evolution

- **3. Centralized VCS** (SVN, CVS)
  - ✓ Team collaboration through central server
  - ❌ Single point of failure
  - ❌ Must be online to commit

# Centralized Version Control Systems

- To **collaborate** with developers on other systems
- These systems (*CVS*, *Subversion*, and *Perforce*) have a single server that contains all the versioned files, and a number of clients that **check out** files from that central place.



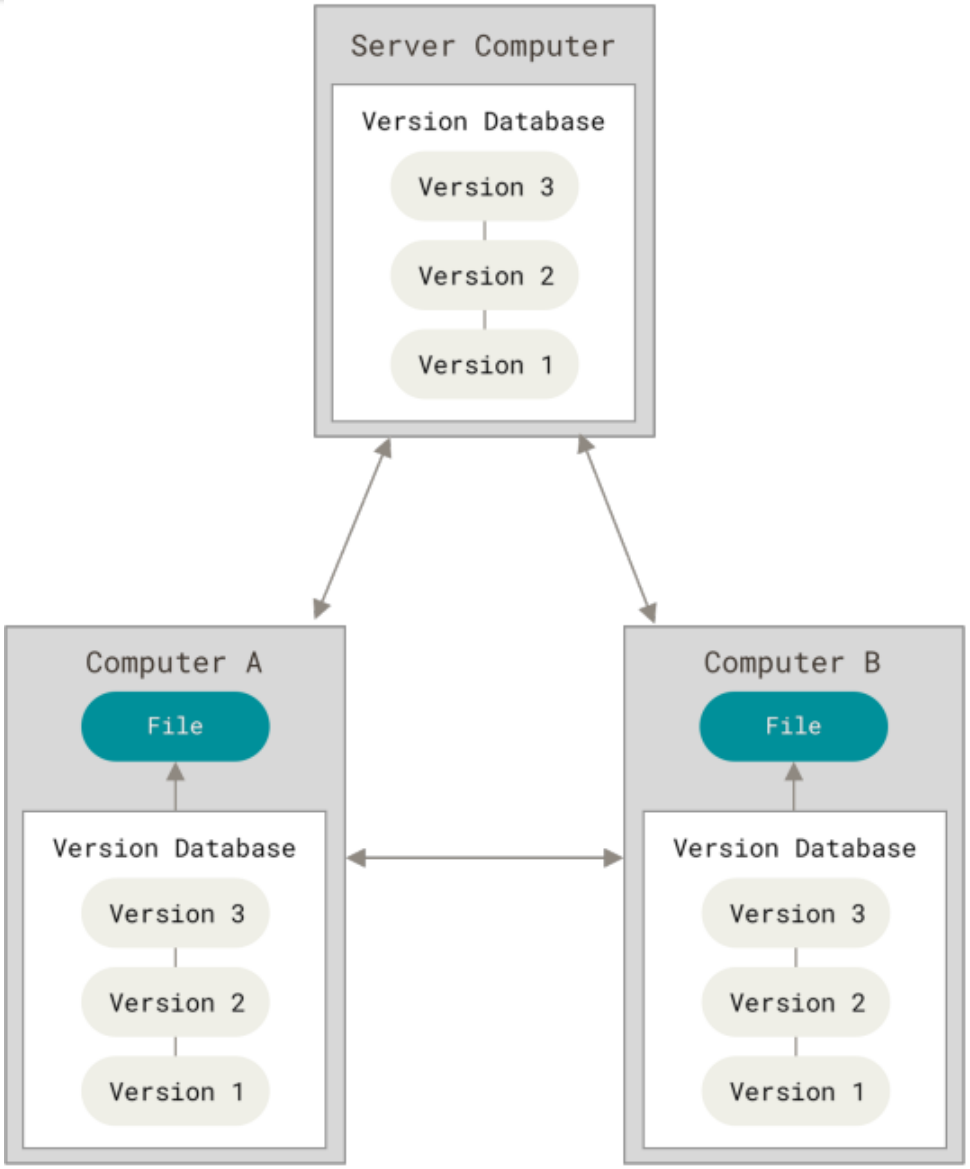
# Centralized Version Control Systems

- **Advantages**
  - Everyone knows to a certain degree *what everyone else on the project is doing*.
  - Administrators have *fine-grained control* over who can do what, and it's far easier to administer a CVCS than local databases on every client.
- **Drawback**
  - *Single point of failure*: If that server goes down for an hour, then during that hour nobody can collaborate at all or save versioned changes to anything they're working on.
  - If the hard disk the central database is on becomes corrupted, and proper backups haven't been kept, you lose absolutely everything

# Version Control Systems Evolution

- **4. Distributed VCS** (Git, Mercurial)
  - ✓ Full history on every computer
  - ✓ Work offline
  - ✓ No single point of failure
  - ✓ Powerful branching and merging

# Distributed Version Control Systems



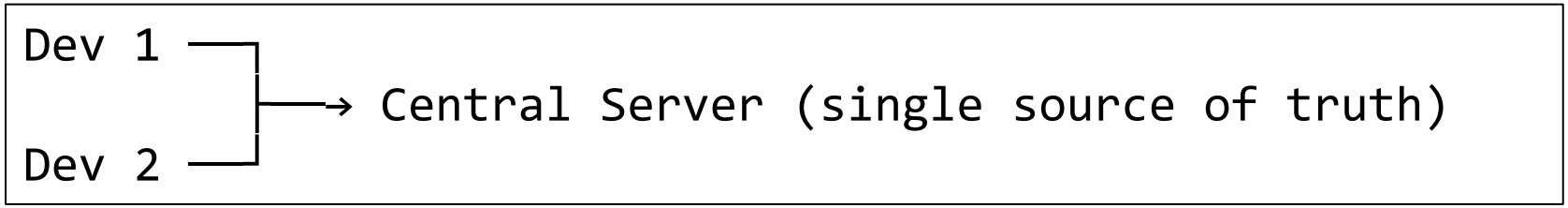
# Distributed Version Control Systems

- In a DVCS (such as **Git**, Mercurial, Bazaar or Darcs), clients don't just check out the latest snapshot of the files; rather, they *fully mirror the repository*, including its full history
- Every clone is really a full backup of all the data.
- Can collaborate with different groups of people in different ways simultaneously within the same project - Hierarchical models

# Centralized vs Distributed VCS

## Understanding the Difference

- **Centralized (SVN, CVS):**




  - Everyone connects to one server
  - Server failure = no work possible
  - Must be online to save versions

# Centralized vs Distributed VCS

## Understanding the Difference

- **Distributed (Git):**

```
Dev 1 (full copy) ⟷ Remote Server ⟷ Dev 2 (full copy)
```

  - Everyone has complete history
  - Work offline, sync later
  - No single point of failure
  - Every clone is a backup

# Introduction to Git

# A Short History of Git

- **Born from Necessity**
- **2005:** Linux kernel development needed a solution
  - Managing thousands of contributors
  - Handling large-scale projects
  - Fast, distributed workflow

# A Short History of Git

**Designed by Linus Torvalds with goals:**

- **Speed** - handles large projects efficiently
- **Simple design** - easy to learn core concepts
- **Powerful branching** - thousands of parallel workflows
- **Fully distributed** - no central dependency
- **Data integrity** - cryptographic verification

# A Short History of Git

**Today:**

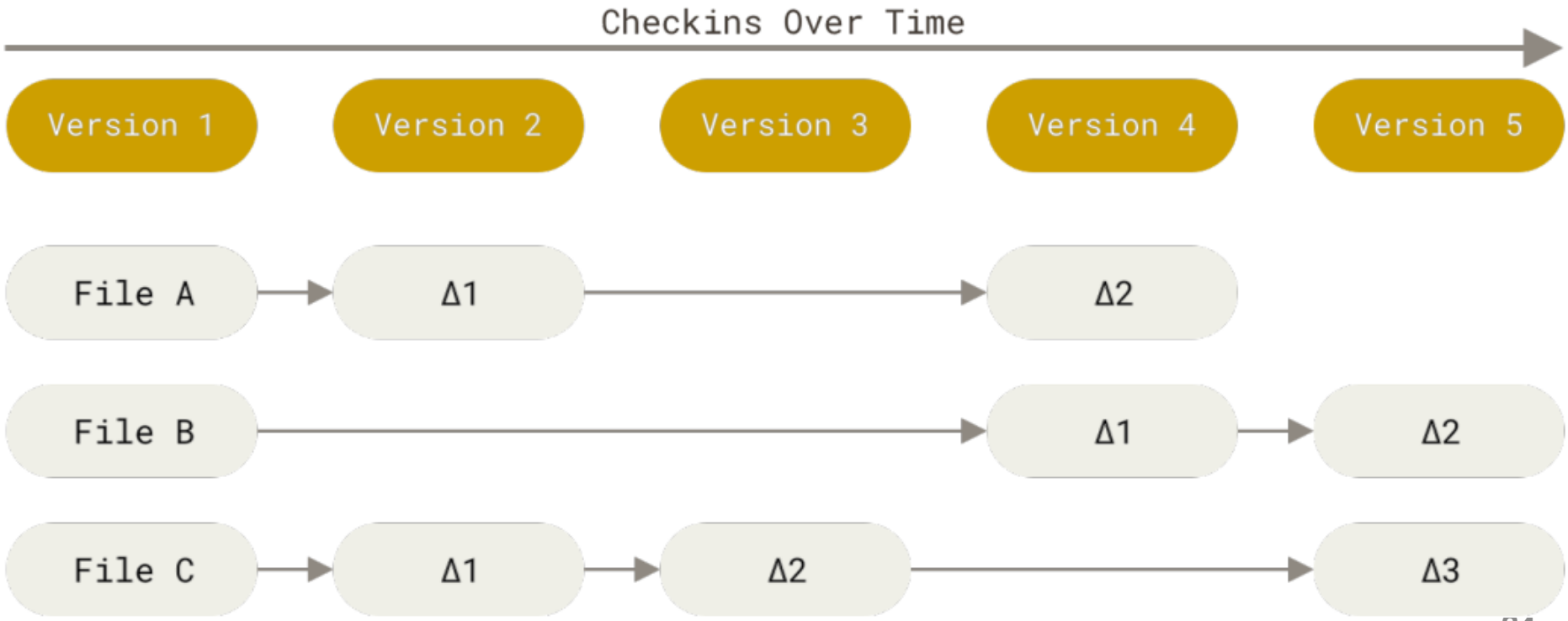
- Industry standard (used by Google, Microsoft, Facebook, etc.)
- 100+ million repositories on GitHub
- Essential skill for data scientists

# What is Git?

- **The Tool Every Data Scientist Needs**
- **Git is:**
  - Free and open-source distributed version control system
  - Industry-standard tool for tracking code changes
  - Enables collaboration at any scale
- **Originally for:** Linux kernel source code
- **Now used for:** Everything from code to data science projects

# Delta-based version control

- Other systems (CVS, Subversion, Perforce) store information as a list of file-based changes.
  - Store as a set of files and the changes made to each file over time (delta-based version control)



# Git (Snapshot-based):

```
Version 1: [Snapshot of entire project]
Version 2: [Snapshot of entire project]
Version 3: [Snapshot of entire project]
```

- Stores complete snapshots (unchanged files = pointers to previous version)
- **Advantages of Snapshots:**
  - Faster operations (no need to reconstruct files)
  - Better data integrity
  - Easier branching and merging
  - More reliable version recovery

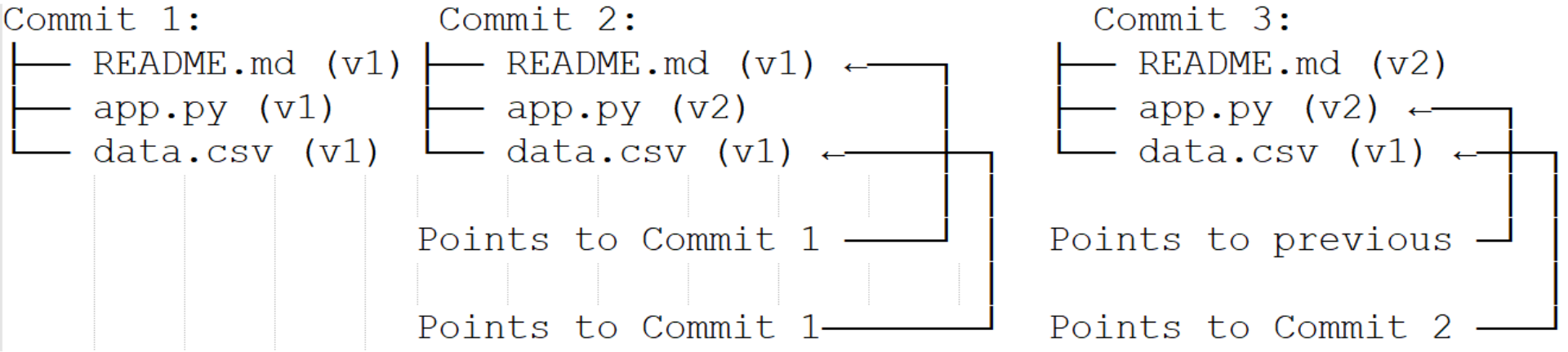
# Git Snapshots

**What Git Actually Stores**

When you make a commit, Git:

- **Takes a snapshot of your entire project** (conceptually)
- **But physically stores it efficiently** using these rules:
  - **Changed files** → stored as new content
  - **Unchanged files** → stored as pointers/references to the previous version

# Git Snapshots




- **Commit 1**: All 3 files stored (first time)
- **Commit 2**: Only app.py changed → only new version of app.py stored
  - README.md and data.csv → pointers to Commit 1's versions
- **Commit 3**: Only README.md changed → only new version of README.md stored
  - app.py → pointer to Commit 2's version
  - data.csv → pointer to Commit 1's version

# Git's Data Model

## Understanding Git's Building Blocks

- **Blob** (Binary Large Object)
  - Represents a file's contents
  - Identified by SHA-1 hash
- **Tree** (Directory)
  - Maps names to blobs or other trees
  - Represents a directory structure

# Git's Data Model

- **Commit** (Snapshot)
  - Points to a tree (project state)
  - Contains metadata: author, date, message
  - Points to parent commit(s)

```
Commit 3fa2b1c
├── author: "Jane Doe"
├── date: "2024-12-06"
├── message: "Add random forest model"
├── tree
│   ├── data/ (tree)
│   ├── models/ (tree)
│   └── train.py (blob)
└── parent: 8e3d4a9
```

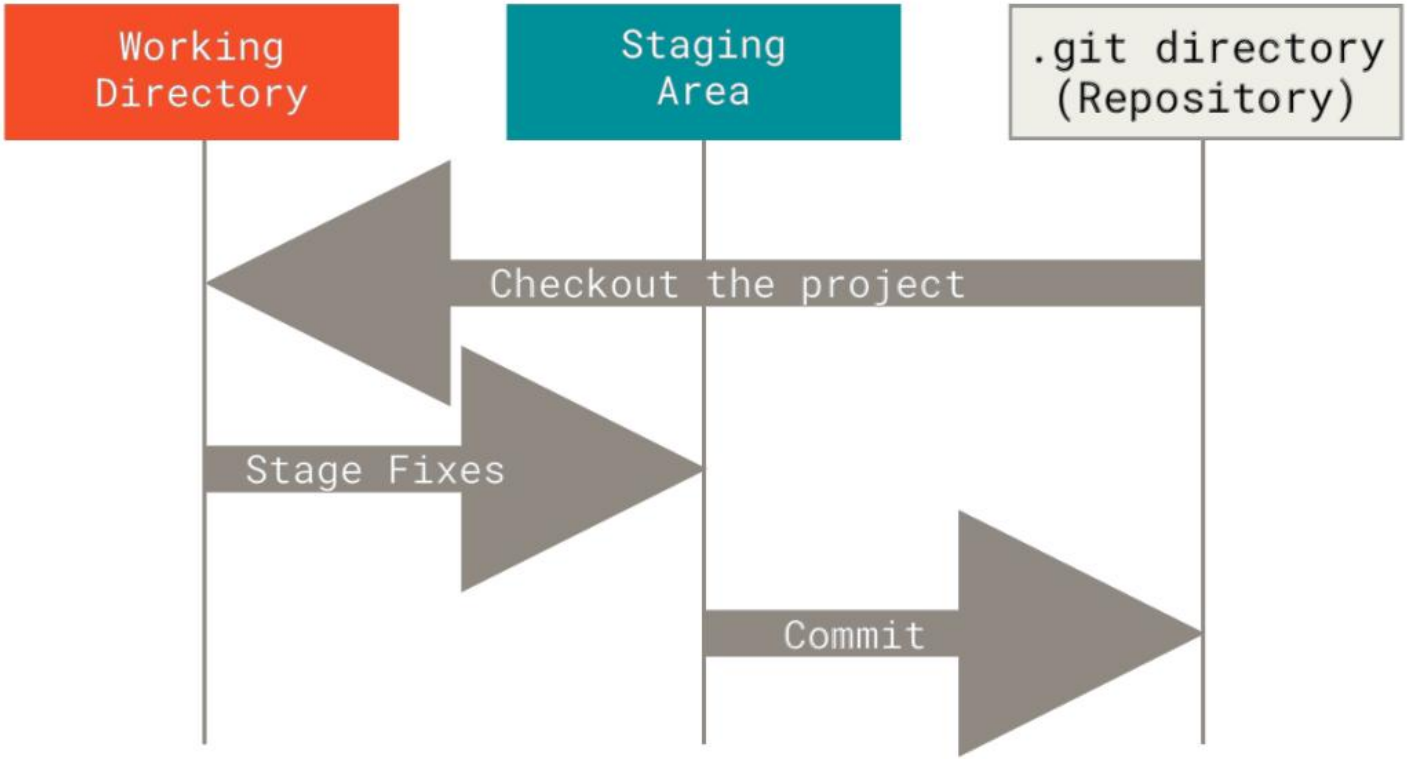
# The Three States of Git

**Three States Your Files Can Be In:**

- **Modified**
  - Changed but not saved to database
  - In your working directory
- **Staged**
  - Marked for next commit
  - In the staging area (index)
- **Committed**
  - Safely stored in Git database
  - Part of project history

# Basic Git: Three stage

- **Working Directory** - your actual files
- **Staging Area** - prepared for commit
- **Git Directory** (.git) - database and metadata



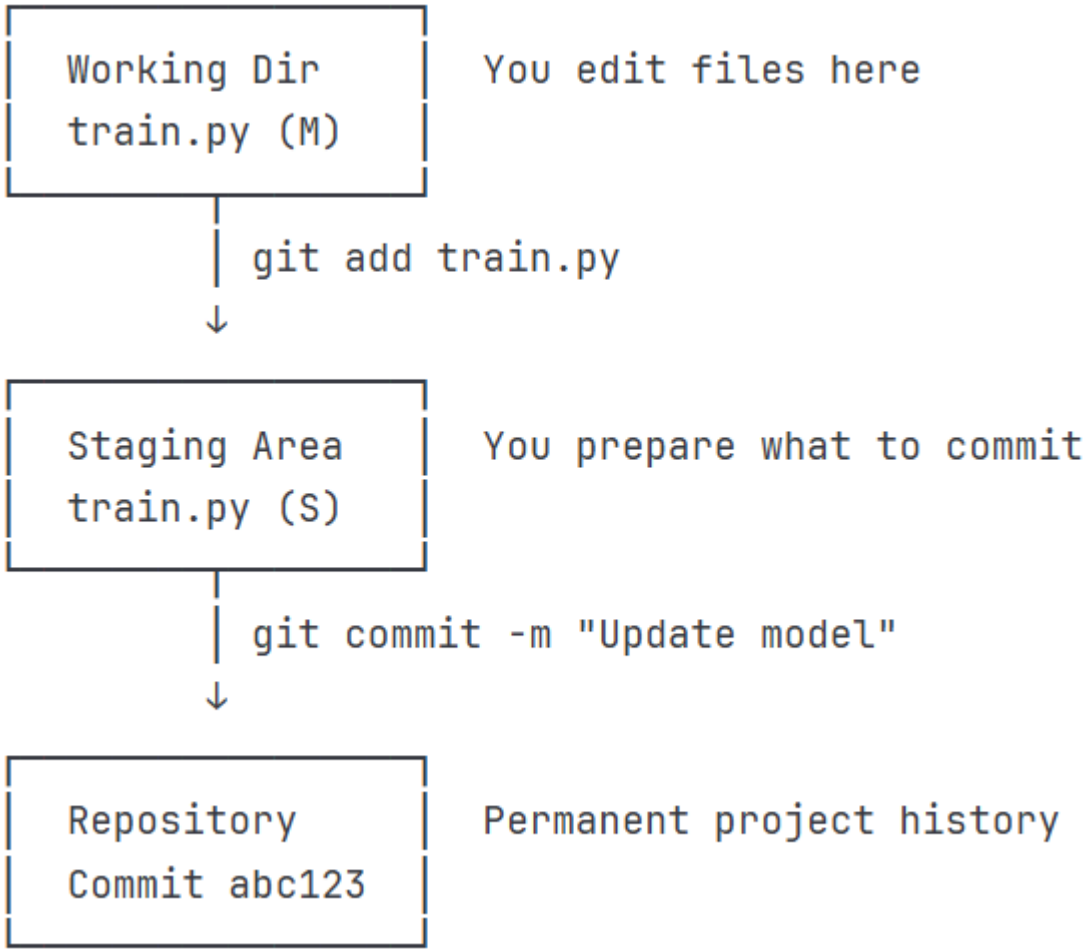
# Basic Git: Three stage

- The **working tree** is a single checkout of one version of the project.
  - These files are pulled out of the compressed database in the Git directory and placed on disk for you to use or modify.
- The **staging area** is a file, generally contained in your Git directory, that stores information about what will go into your next commit.
- The **Git directory** is where Git stores the metadata and object database for your project.
  - This is the most important part of Git, and it is what is copied when you clone a repository from another computer.

# Basic Git: Three stage

**Why Staging Area?**

- Selectively choose what to commit
- Create logical, focused commits
- Review changes before committing



# Basic Git workflow

1. **Modify** files in your working directory

```
Edit: analysis.py, model.py, README.md
```

2. **Stage** the changes you want to commit

```
git add analysis.py model.py
   # README.md not staged - will commit later
```

3. **Commit** to save snapshot permanently

```
git commit -m "Implement feature engineering pipeline"
```

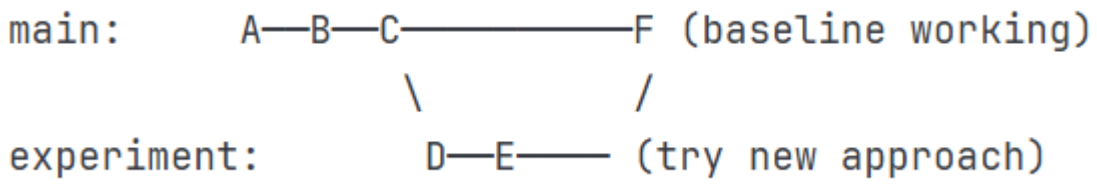
# Basic Git workflow

**Key Principle:** Each commit should be a *logical unit of work*

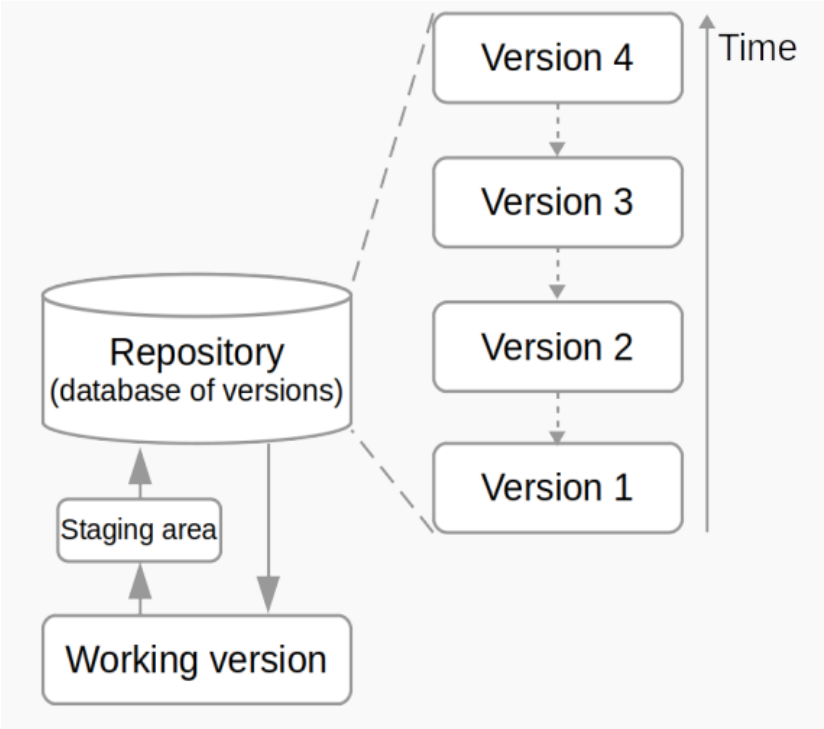
- Not too big (multiple unrelated changes)
- Not too small (incomplete changes)
- Focused on one task or fix

# Branching

- **Scenario:** You have a working baseline model
  - Want to experiment with deep learning
  - Don't want to break the working model
  - Need to compare results
- **Solution**: Create a Branch!

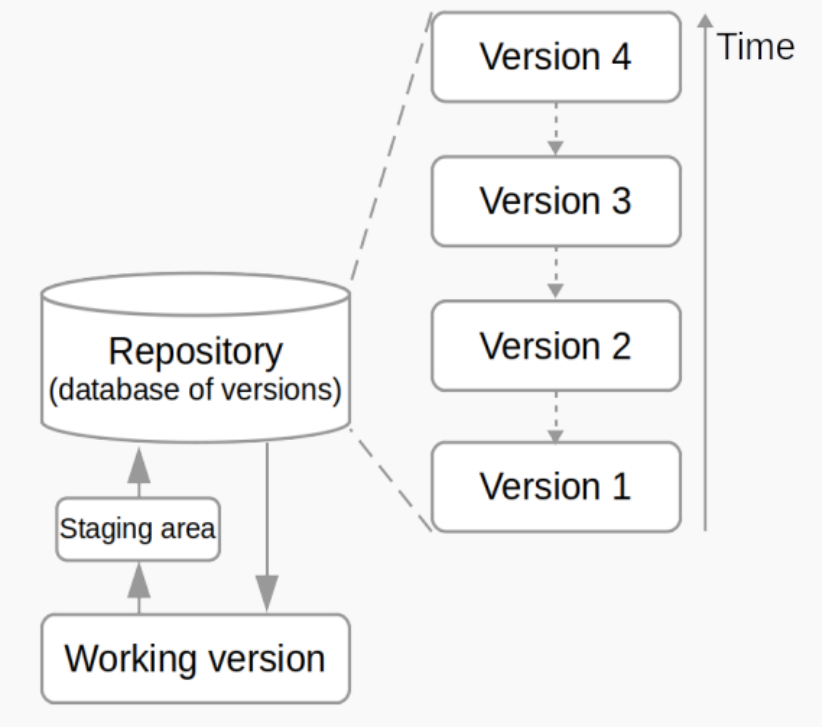


# How git work



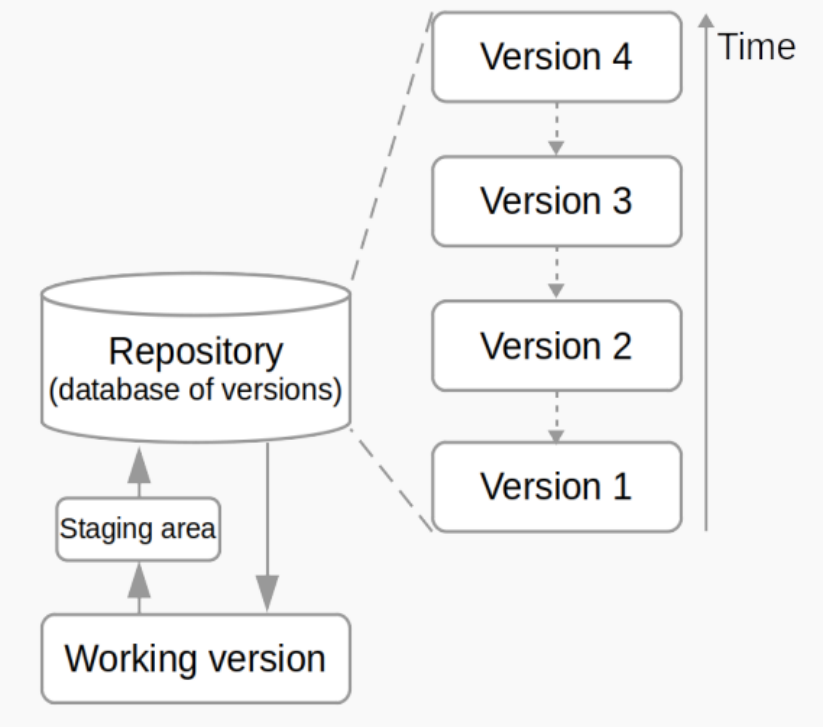

- Version 4 pointing to version 3 means: version 4 is created by modifying version 3
- Working version (working directory/tree) is a version we can modify to create a new version

# How git work



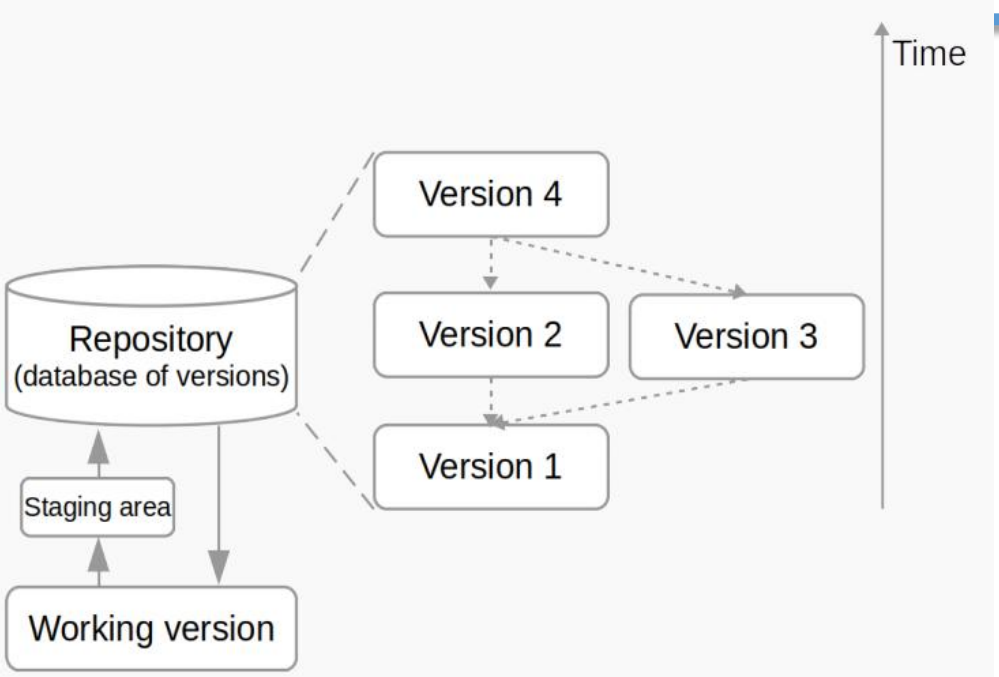

- After modifying the working version, we will save the new version to the repository
- Why do we need to go through the staging area?
  - Because we may make many changes to the working version, but we only want to include some changes to the new version

# How git work




- Versions can be created in a simple way: version 2 is created from version 1, version 3 is created from version 2, … → there is only one branch

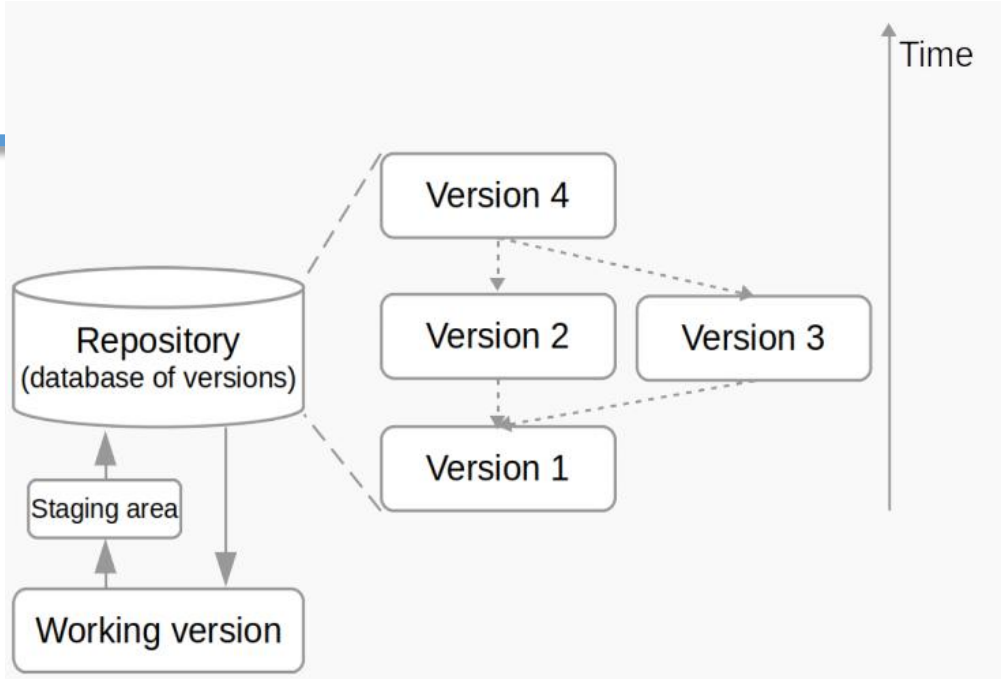
# How git work




- Git allow creating branches from the main branch; e.g., we are coding an app, and we want to code and experiment a feature without interfering with the current code in the main branch

➔Create a second branch to code and experiment; when everything is OK we can merge this branch into the main

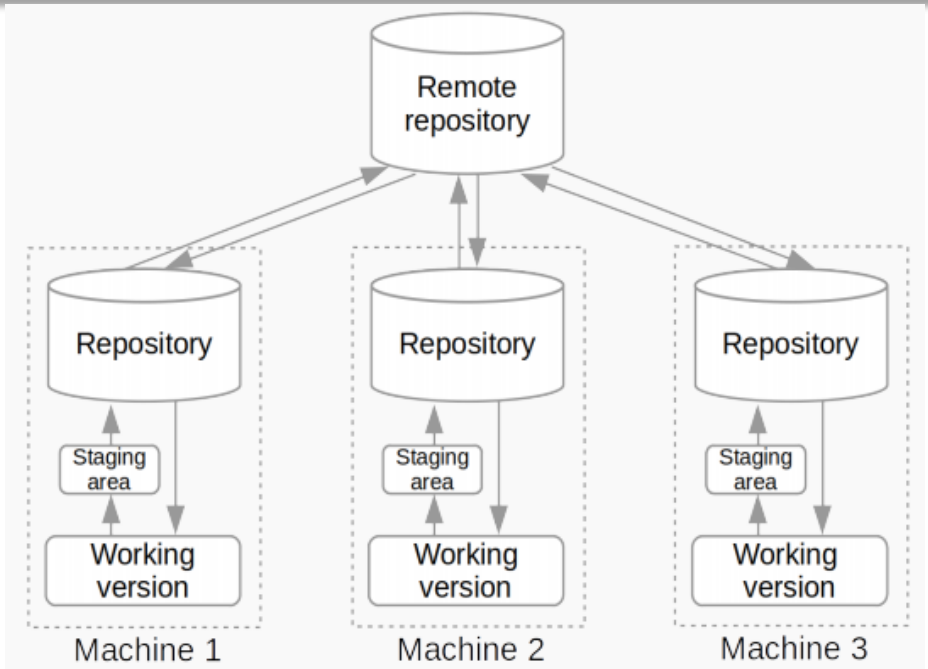
# How git work




- Git allow creating branches from the main branch; e.g., we are coding an app, and we want to code and experiment a feature without interfering with the current code in the main branch

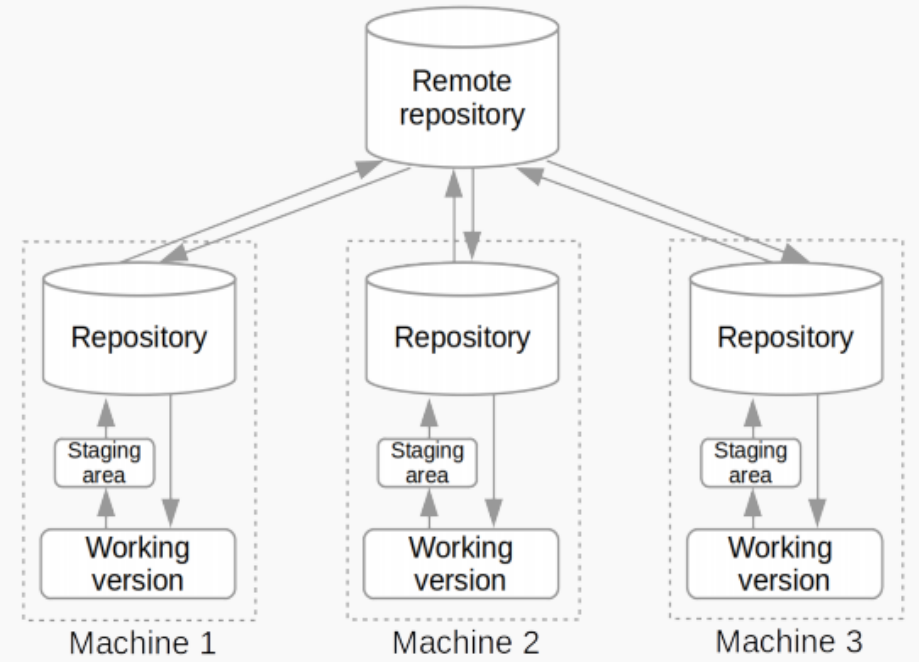
➔Create a second branch to code and experiment; when everything is OK we can merge this branch into the main branch, otherwise we can discard this branch

# How git work




- **Git** supports collaboration through a remote repository (**Github** is a service providing remote repositories for Git)
- When working solo, we may still want to use remote repositories to backup data, to access from other computers

# How git work




- Git is a distributed version control tool: each member in a team will have a local repository
- With central version control tools, data is stored in one place — the remote repository; if data in this place is loss then …

# How git work

**Push & Pull:**

- **Push**: Send your commits to remote
- **Pull**: Get updates from remote
- **Clone**: Copy entire repository

**Why Remote Repository?**

- Backup your work
- Share with team
- Access from multiple devices
- Enable code review
- Showcase your portfolio

# Git - Basic usage

# Installation & Setup

- 1. Verify Installation

```
git --version
# Output: git version 2.40.0 or higher
```

- 2. Configure Identity

```
git config --global user.name "Your Name"
git config --global user.email
"your.email@university.edu"
```

- 3. Set Default Branch Name

```
git config --global init.defaultBranch main
```

- 4. Verify Configuration

```
git config --global init.defaultBranch main
```

# Git Repository

## Two Ways to Start

- **Method 1: Initialize Existing Project**

```
cd my-ml-project
git init
# Creates .git directory
```

- **Method 2: Clone Existing Repository**

```
git clone https://github.com/username/awesome-ds-
project.git
cd awesome-ds-project
```

# Git Repository

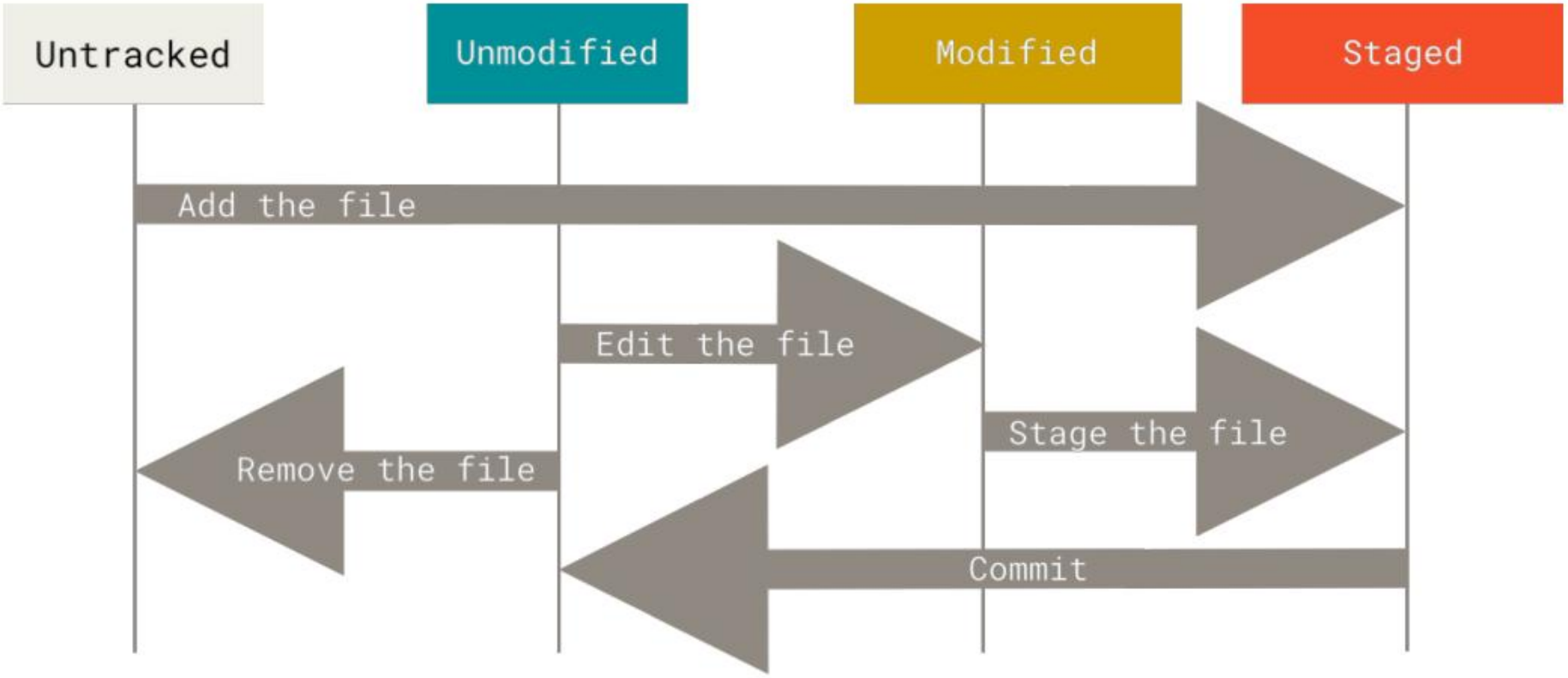
## What Just Happened?

- .git/ folder created (Git's brain)
- Repository ready to track changes
- All files are "untracked" initially

## First Steps After Init:

```
git status  # See what's happening
git add .   # Stage all files
git commit -m "Initial commit"  # First snapshot!
```

# Initializing a Repository



# Essential Git Commands - Basics

- Status & Information:

```
git status                  # What's changed?
git log                     # Commit history
git log --oneline           # Compact history
git log --graph             # Visual history
```

- Making Changes:

```
git add <file>              # Stage specific file
git add .                   # Stage all changes
git commit -m "msg"         # Create commit
git commit -am "msg"        # Add & commit tracked files
```

# Essential Git Commands - Basics

- Viewing Differences:

```
git diff                    # Unstaged changes
git diff --staged           # Staged changes
git diff HEAD               # All changes from last commit
```

- Getting Help:

```
git help <command>          # Open manual
git <command> --help        # Same thing
```

# Essential Git Commands - Branching

- Creating & Switching:

```
git branch                          # List branches
git branch feature-xyz              # Create branch
git checkout feature-xyz            # Switch to branch
git checkout -b feature-xyz         # Create + switch
# Newer syntax:
git switch feature-xyz              # Switch branch
git switch -c feature-xyz           # Create + switch
```

# Essential Git Commands - Branching

- Merging:

```
git checkout main          # Switch to main
git merge feature-xyz      # Merge feature into main
git branch -d feature-xyz  # Delete merged branch
```

- Viewing Branches:

```
git branch -a              # All branches (local + remote)
git log --all --graph --oneline # Visual branch history
```

# Essential Git Commands - Remotes

- Setup Remote:

```
git remote add origin https://github.com/user/repo.git
git remote -v                    # View remotes
```

- Pushing & Pulling:

```
git push origin main             # Push to remote
git push -u origin main          # Push + set upstream
git pull origin main             # Get updates
git fetch origin                 # Download without merging
```

# Essential Git Commands - Remotes

- Sync Workflow:

```
git pull                          # Get latest changes
# ... do work ...
git add .
git commit -m "Add feature"
git push                          # Share your work
```

- Clone:

```
git clone <url>                   # Copy remote repository
```

# Remotes

- git remote: list remotes
- git remote add <name> <url>: add a remote
- git push <remote> <local branch>:<remote branch>: send objects to remote, and update remote reference
- git branch --set-upstream-to=<remote>/<remote branch>: set up correspondence between local and remote branch
- git fetch: retrieve objects/references from a remote
- git pull: same as git fetch; git merge
- git clone: download repository from remote

# Essential Git Commands - Undo

```
git commit --amend #edit a commit’s contents/message
git reset HEAD <file> #unstage a file
git checkout -- <file> #discard changes
```

# The .gitignore File

- **What NOT to Track**
- **Why .gitignore?**
  - Keep repository clean
  - Avoid committing sensitive data
  - Prevent large file issues
  - Ignore generated files

# The .gitignore File

- **What NOT to Track**
- **Why .gitignore?**
  - Keep repository clean
  - Avoid committing sensitive data
  - Prevent large file issues
  - Ignore generated files

# Github

# What is Github

- **Git's Home in the Cloud**
- **GitHub is NOT Git:**
  - Git = version control tool (runs locally)
  - GitHub = hosting service for Git repositories

# What is Github

- **GitHub Provides:**
  - Cloud storage for repositories
  - Collaboration tools
  - Issue tracking
  - Code review (Pull Requests)
  - Project management
  - CI/CD (GitHub Actions)
  - Documentation (Wiki, README)
  - Portfolio showcase

# What is Github

- **Alternatives:**
  - GitLab (more DevOps features)
  - Bitbucket (Atlassian ecosystem)
  - Azure DevOps
- **For Data Scientists:**
  - Share projects publicly
  - Collaborate on research
  - Build professional portfolio
  - Contribute to open source

# GitHub Workflow - Forking

- **Contributing to Projects**
- **What is Forking?**
  - Creating your own copy of someone else's repository
  - You can freely experiment without affecting original
  - Foundation of open source contribution

# GitHub Workflow - Forking




- Fork Workflow:

```
Original Repo (owner)
        ↓ (Fork)
Your Fork (your account)
        ↓ (Clone)
Local Copy (your computer)
```

- **When to Fork:**
  - Want to contribute to open source project
  - Don't have write access to original
  - Want to experiment with someone's code
  - Building on existing project

# The GitHub Flow

1. **Fork** the project on GitHub
2. **Clone** your fork locally

```
git clone https://github.com/YOUR-USERNAME/forked-repo.git
```

3. **Create** a topic branch from main

```
git checkout -b fix-data-loader
```

4. **Make** commits to improve the project

```
git add data_loader.py
git commit -m "Fix null handling in data loader"
```

# The GitHub Flow

5. **Push** branch to your GitHub fork

```
git push origin fix-data-loader
```

6. **Open** a Pull Request on GitHub
7. **Discuss** and continue committing if needed
8. **Wait** for maintainer to merge or provide feedback
9. **Sync** updated main back to your fork

```
git pull upstream main
```

# Pull Requests - Code Review

- **What is a Pull Request (PR)?**
  - Proposal to merge your changes into another repository
  - Platform for discussion and code review
  - Quality control mechanism

# Pull Requests - Code Review

- **Anatomy of a Good PR:**
  - Clear title and description
  - Focused on one feature/fix
  - Tests included
  - Documentation updated
  - Screenshots (if UI changes)
  - Links to related issues

# Keeping Your Fork Updated

- **Staying in Sync with Original**
- **Why Keep Updated?**
  - Get latest features
  - Avoid merge conflicts
  - Contribute with current code

# Keeping Your Fork Updated

- Setup (once):

```
# Add original repo as 'upstream'
git remote add upstream https://github.com/ORIGINAL/repo.git
git remote -v  # Verify
```

# Keeping Your Fork Updated

- Update Process:

```
# 1. Fetch latest from original
git fetch upstream
# 2. Switch to your main branch
git checkout main
# 3. Merge upstream changes
git merge upstream/main
# 4. Push to your fork
git push origin main
# 5. Update your feature branch
git checkout your-feature-branch
git merge main  # Resolve any conflicts
```

# Managing Jupyter Notebooks

- **The Problem:** Notebooks are JSON → messy Git diffs
- **Solution 1: nbdime** (Notebook Diff and Merge)

```
pip install nbdime
nbdime config-git --enable --global
```

# Managing Jupyter Notebooks

- **The Problem:** Notebooks are JSON → messy Git diffs
- **Solution 2:** Clear Outputs Before Committing

```
pip install nbstripout
# Strip outputs from notebook
nbstripout notebook.ipynb

# Auto-strip on every commit
nbstripout --install
```

# Managing Jupyter Notebooks

- **The Problem:** Notebooks are JSON → messy Git diffs
- **Solution 3:** Jupytext

```
pip install jupytext

# Convert notebook to Python script
jupytext notebook.ipynb --to py

# Track .py file instead of .ipynb
```

# Github starter

- https://docs.github.com/en/get-started/quickstart/hello-world
- https://docs.github.com/en/get-started/quickstart
- https://github.com/skills/introduction-to-github

# Important reminder

- Use Git & Github to do version control and data backup for your code files
- **Note**: you should be thoughtful about whether you should make repositories in Github public

# Reference

- https://missing.csail.mit.edu/2020/version-control/
- https://git-scm.com/book/en/v2




# THE END